

Proteo User Manual

Configurable Synthetic Application for Studying Malleability in HPC

July 28, 2026

Contents

Quick Install	3
1 Introduction	3
1.1 Terminology	3
1.2 Configuration format	4
1.3 Manual organization	4
2 Architecture	4
2.1 SAM modules	5
2.2 Computation stages	5
2.3 Communication stages	7
2.4 I/O stages	7
2.5 MaM resize pipeline	7
2.6 General working	9
2.7 FactorS and scalability	9
2.8 MaM as a standalone library	10
3 Installation	10
3.1 Prerequisites	10
3.2 Obtain the source	10
3.3 Compile	10
3.4 Repository layout	11
3.5 Verify the installation	11
4 Running Proteo	12
4.1 Single execution with automatic allocation	12
4.2 Execution with singleRunCostum.sh	13
4.2.1 Local or personal computer	13
4.2.2 Cluster with Slurm	13
4.2.3 Multiple machines without an RMS	13
4.3 Combinatorial JSON expansion	14

4.4	Error checking	14
4.5	Intermediate output files	15
5	JSON configuration reference	15
5.1	General parameters	16
5.2	Phase and stage parameters	16
5.2.1	Stage types	17
5.2.2	Stage validation rules	17
5.3	Group parameters	18
5.3.1	Group termination semantics	18
5.3.2	Spawn strategies	18
5.3.3	Redistribution methods and strategies	19
5.3.4	Strategy combination rules	19
5.3.5	Other group validation rules	19
5.4	Full example	20
5.5	Modeling communication and computation	20
6	GenConfig	20
6.1	Install and launch	20
6.2	Editor workflow	20
6.3	Single vs Multi mode	23
6.4	Combinatorial variant syntax	23
6.5	Expanding complex JSON	24
6.6	Strategy legend in the UI	24
6.7	Related files	24
7	Examples	24
7.1	Minimal single run	25
7.2	Custom allocation	25
7.3	Creating a config with GenConfig	25
7.4	Simple resize example	25
7.5	Multi-phase application	26
7.6	Combinatorial study	26
7.7	Conjugate-gradient-like profile	27
7.8	Error checking after a batch	27
8	Output analysis	27
8.1	Analysis scripts	27
8.2	Notebook analyser (WIP)	28
8.3	Recommended practice	28
A	Output CSV files format	28

Quick Install

This page summarizes the shortest path from clone to a first run. See Section 3 for prerequisites and build options.

```
git clone http://lorca.act.uji.es/gitlab/martini/malleability_benchmark.  
git  
cd malleability_benchmark  
  
cd Codes  
make MAM_USE_SLURM=1  
  
cd ../Results  
bash ../Exec/singleRun.sh ../Codes/test.json mypartition 0 1 0
```

Listing 1: Minimal Proteo setup and smoke test.

Replace `mypartition` with a valid Slurm partition. On success, output files such as `R0.Global.out` appear in the current directory.

- JSON parameter reference: Section 5
- Web editor for JSON files: Section 6
- Execution scripts and outputs: Section 4

1 Introduction

Proteo is a configurable synthetic application for studying *malleability* in high-performance computing (HPC). It emulates the behaviour of iterative MPI scientific applications that can be resized on-the-fly (expanded, shrunk, or migrated) while running under a resource manager.

Proteo is composed of two C/MPI modules:

- **SAM** (Synthetic Application Module): emulates application **computation, communication**, and **I/O** through configurable *phases* and *stages*.
- **MaM** (Malleability Module): performs dynamic process management, spawn/shrink operations, and data redistribution between source and target process groups.

Proteo focuses on **iterative MPI applications** whose iterations combine compute, MPI communication, and I/O stages. With the appropriate JSON configuration, it can approximate a wide range of MPI application profiles.

1.1 Terminology

During a resize operation, the existing process group is referred to as *sources*; the newly created group is the *targets*. Depending on the spawn method and

strategy, some source ranks may also join the target group. In all cases, the target group continues the emulated application execution.

A **phase** is a segment of the emulated application: it runs for a fixed number of iterations (**Total_Iters**) and contains an ordered list of **stages** (computation, MPI communication, or I/O). Phases are executed sequentially within a process group.

A **group** defines one segment of the overall malleable execution: how many MPI processes run, for how many iterations, and which MaM spawn/redistribution options apply before the next resize (if any). The number of groups is **Total_Resizes** + 1.

1.2 Configuration format

Proteo is configured through **JSON files**. Each file describes:

1. Global simulation parameters (**general**).
2. One or more application phases with nested stages (**phases**).
3. One process group per resize boundary (**groups**).

The web tool **GenConfig** (Section 6) helps create and validate these files. Section 5 documents every parameter.

1.3 Manual organization

- Section 2: SAM/MaM architecture and execution model.
- Section 3: prerequisites and build instructions.
- Section 4: execution scripts and output files.
- Section 5: JSON parameter reference.
- Section 6: GenConfig web editor and combinatorial expansion.
- Section 7: worked examples.
- Section 8: post-run analysis.
- Appendix A: CSV output format for custom analysis.

2 Architecture

Proteo emulates a malleable MPI application through five cooperating modules inside SAM, backed by MaM for resize operations.

2.1 SAM modules

1. **Initialization:** Rank 0 of the first group reads the JSON configuration from disk and builds the in-memory **configuration** structure. That structure is broadcast to every rank in the group and, after each resize, from the source root to target ranks. During initialization SAM also calibrates derived parameters (per-stage operation counts, communication volumes, I/O sizes) from the JSON values and the hardware where the job runs. This module runs once at job start and again for each target group after spawn and data redistribution.
2. **Application emulation:** SAM executes the configured **phases** in order. For each phase it repeats **Total.Iters** iterations; within each iteration every **stage** runs sequentially. Stages may emulate CPU-bound or memory-bound computation, blocking or non-blocking MPI communication, or parallel I/O. Timings are recorded per stage and per iteration for later output.
3. **Resize:** When a non-final group completes its allotted work, MaM is invoked to spawn or merge target processes and redistribute application state from sources to targets. Depending on spawn strategy, sources may continue executing iterations while spawn or asynchronous redistribution proceeds in the background.
4. **Monitoring:** Every stage, iteration, spawn, and redistribution interval is measured with **MPI.Wtime**. Optional barriers (JSON **Rigid** and compile-time **MAM.USE_BARRIERS**) tighten timing accuracy at the cost of overhead.
5. **Completion:** When a group finishes its role in the emulation, SAM writes intermediate result files containing configuration echo, per-stage timings, and malleability metrics (T_{spawn} , T_{SR} , T_{AR} , etc.).

Figure 1 shows the overall control flow. Listing 2 sketches the corresponding program structure.

Each **phase** contributes **Total.Iters** iterations; within each iteration all **stages** of that phase execute sequentially. When all phases of a group finish, a resize occurs unless this is the last group (see Section 5.3.1).

2.2 Computation stages

Compute stages drive CPU or memory workload without MPI messaging:

- **Type 0 (Monte Carlo π):** Repeated pseudo-random sampling to estimate π . The kernel is CPU-bound and does not perform large memory traversals. **Granularity** controls the sample count per operation; SAM repeats operations until the configured **Stage.Time** is reached under the current **FactorS** and process count.

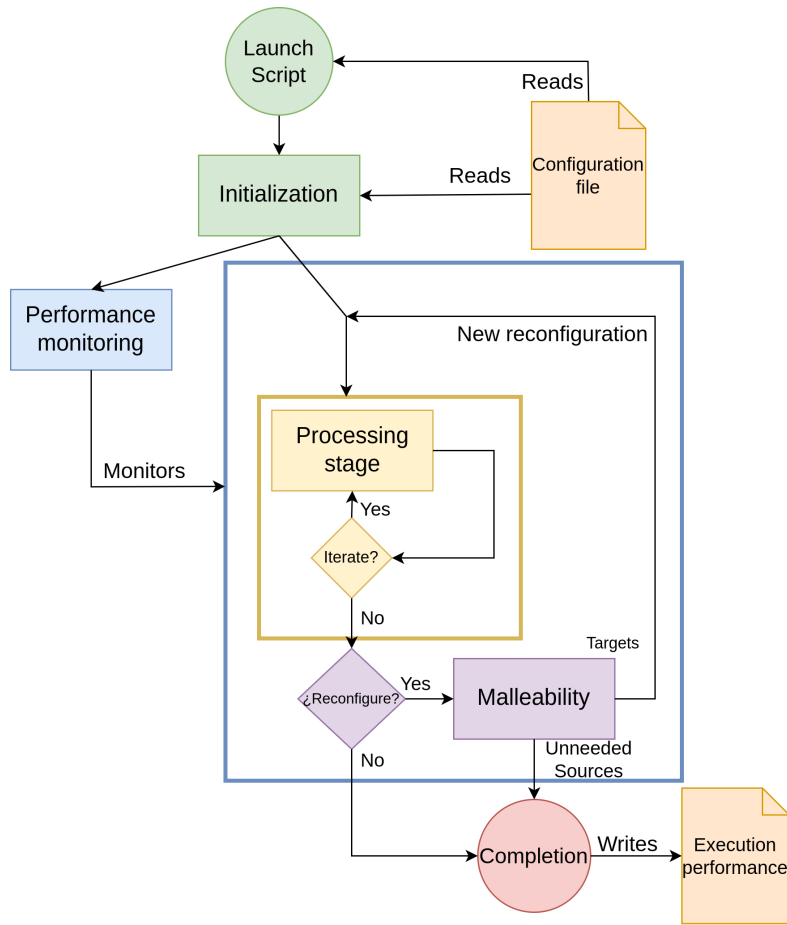


Figure 1: Flowchart of the synthetic application modules.

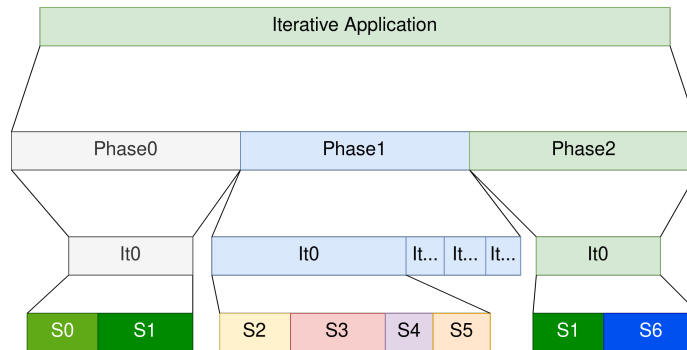


Figure 2: Phase-based iteration model: each phase defines its own stage sequence and iteration count.

- **Type 1 (matrix–vector):** Column-major matrix–vector multiplication. The kernel is memory-bandwidth bound. As with type 0, **Granularity** sets the problem size and **Stage.Time** sets the target duration after scaling.

For both types, **Stage.Bytes** is unused. SAM derives an internal operation count from the measured baseline time T_{op} at the current **Granularity**, the configured **Stage.Time**, and the group **FactorS**. After a resize, T_{op} and the operation count are recomputed if **FactorS** or **Procs** changes (Section 2.7).

2.3 Communication stages

Communication stages exercise MPI patterns among ranks in the active group:

- **Point-to-point (2):** **MPI_Sendrecv** between ranks on neighbouring nodes following a ring-like neighbour assignment.
- **IPoint-to-point (3):** Non-blocking send/receive posted with **Stage_Identifier**. Must be paired with a Wait stage (type 4) that completes the outstanding requests.
- **MPI_Wait (4):** Completes non-blocking operations for a given **Stage_Identifier**. One Wait stage may complete **multiple** IPoint stages that share the same identifier.
- **Bcast (5), Allgather (6), Reduce (7), Allreduce (8):** Standard MPI collectives over the group communicator.

Each communication stage must specify either **Stage.Bytes** > 0 (preferred: explicit message volume) or **Stage.Time** > 0 (timed emulation from which SAM derives bytes using network calibration). Non-compute granularity rules apply when bytes are absent or when time-capping is enabled; see Section 5.2.2.

2.4 I/O stages

- **Write (9) and Read (10):** Emulated parallel I/O. Each stage requires **Stage.Bytes** > 0 or **Stage.Time** > 0, plus **Stage.Involved_Procs** (0 means all ranks participate). These stages let Proteo model applications whose iterations include checkpointing, restart, or dataset movement alongside compute and MPI traffic.

2.5 MaM resize pipeline

A resize involves three steps:

1. Signal availability to the resource manager (simulated via configuration when Slurm integration is enabled).
2. Create the target MPI process group (*spawn*).

3. Redistribute data from sources to targets (*redistribution*).

Spawn methods:

- **Baseline (0):** Create Y target processes and remove the X sources (MPI.Spawn).
- **Merge (1):** Create or remove $|Y - X|$ processes and merge communicators where possible.

Spawn strategies (combinable indices in `Spawn_Strategy`, except where noted):

- **1 Pthread:** asynchronous spawn in a background thread.
- **2 Single:** only rank 0 spawns.
- **3 Intercomm:** intercommunicator-based spawn.
- **4 Multiple / 5 Parallel:** multi-source strategies; **mutually exclusive** with each other. **Parallel** coordinates process creation with the RMS so expand and shrink operations keep allocations consistent with RMS rules (jobs do not violate resource-manager constraints when resizing).

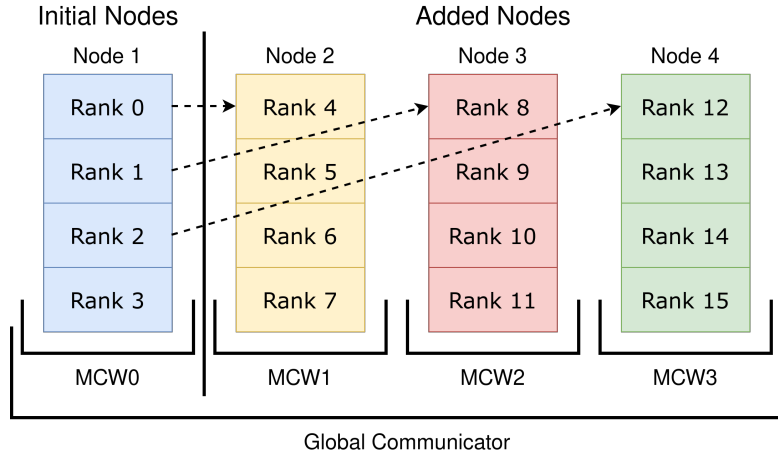


Figure 3: Overview of parallel spawn strategies in MaM.

Physical mapping (Dist):

- **spread:** spread processes across as many nodes as possible.
- **compact:** fill nodes before using the next node.


```

int main() {
    if (init) {
        if (rank == 0) read_config();
        broadcast_config();
        calculate_derived_parameters();
    } else {
        receive_config_from_sources();
        redistribute_data();
    }
    for (phase = 0; phase < n_phases; phase++)
        for (iter = 0; iter < phase_iters; iter++)
            for (stage = 0; stage < n_stages; stage++)
                process_stage(stages[stage]);
    if (!last_group) {
        spawn_targets();
        redistribute_data_to_targets();
    }
    write_output_files();
}

```

Listing 2: Basic skeleton of the synthetic application.

Data redistribution splits the payload into synchronous (SDR) and asynchronous (ADR) **element counts**. The actual byte volume moved is $\text{SDR} \times \text{Datasize}$ and $\text{ADR} \times \text{Datasize}$. Constant data may be sent asynchronously while sources continue computing; variable data is sent synchronously. Methods include all-to-all among sources (0), point-to-point (1), and RMA Lock / LockAll (2–3). Spawn and redistribution **strategy combination rules apply when $\text{ADR} > 0$** (Section 5.3.4).

2.6 General working

Listing 2 shows a simplified view of SAM’s main control flow. Source ranks read and broadcast the configuration, then derive stage parameters. Target ranks receive the configuration from sources and complete data redistribution. All ranks then walk phases, iterations, and stages. If more groups remain, MaM spawns targets and redistributes state; otherwise the group writes its output files and finishes.

2.7 FactorS and scalability

FactorS emulates how compute-stage duration scales with the process count. Values above 1 increase effective stage time; values below 1 decrease it. For an ideal speedup from 2 to 4 processes with sequential $\text{Stage_Time} = 4$ s, **FactorS** would be 0.5 and 0.25 respectively.

Non-direct parameters T_{op} (time per operation at current granularity) and **operations** (repeat count to reach the configured **Stage_Time** under **FactorS**) are recomputed after each resize when **FactorS** changes.

2.8 MaM as a standalone library

MaM can be linked into **other MPI applications** to add malleability (spawn, shrink, redistribution) without using SAM. Its public interface, configuration keys, and integration workflow will be documented in a **separate MaM manual**. This Proteo manual covers only the SAM+MaM benchmark path.

3 Installation

This section describes how to obtain, compile, and verify a Proteo installation.

3.1 Prerequisites

Required:

- GNU C compiler.
- **MPICH** 3.x or 4.0.x with OFI or Nemesis netmod. Dynamic process creation has been tested with MPICH. The UCX netmod has only been tested with MPICH 5.0.0.
- **Slurm** workload manager (tested with Slurm-wlm 19.05.5). Slurm is optional for compilation, but the default execution scripts assume it for job submission.

Optional:

- Python 3.8+ with NumPy and Pandas (post-run analysis only).
- Flask (for GenConfig; see Section 6).

3.2 Obtain the source

Clone the repository:

```
git clone http://lorca.act.uji.es/gitlab/martini/malleability_benchmark.  
git  
cd malleability_benchmark
```

Listing 3: Clone Proteo.

3.3 Compile

From the repository root, build with **make** in **Codes/**. Three optional flags control MaM behaviour (**Codes/Makefile**):

Flag	Default	Meaning
MAM_USE_SLURM	0	Set to 1 to enable Slurm/RMS integration in MaM (required for normal Proteo runs with <code>singleRun.sh</code>).
MAM_USE_BARRIERS	0	Set to 1 to insert additional barriers inside MaM for stricter timing (may slow execution). Distinct from JSON Rigid.
MAM_DEBUG	0	Debug verbosity at compile time; integer 0 (off) through 6 (most verbose). Values greater than 0 also enable compiler flags <code>-g</code> and <code>-O1</code> .

Table 1: MaM compile-time flags passed through `Codes/Makefile`.

Typical Slurm-enabled build:

```
cd Codes
make MAM_USE_SLURM=1
```

Listing 4: Build MaM and SAM with Slurm support.

Optional debug or barrier build:

```
make MAM_USE_SLURM=1 MAM_USE_BARRIERS=1 MAM_DEBUG=3
```

This compiles the MaM shared library first, then links SAM (`Codes/SAM/build/a.out`). A configuration stub is written to `Codes/build/config.txt` and sourced by execution scripts.

To remove build artifacts:

```
cd Codes && make clean
```

3.4 Repository layout

- `Codes/`: MaM and SAM source code and binaries.
- `Exec/`: Slurm launch scripts and Python helpers.
- `GenConfig/`: JSON configuration web editor.
- `Results/`: default directory for experiment outputs.
- `Analysis/`: Python scripts for post-processing.

3.5 Verify the installation

A minimal smoke test uses the sample JSON in `Codes/test.json`. From the directory where you want output files (e.g. `Results/`):

```
bash ../Exec/singleRun.sh ../Codes/test.json mypartition 0 1 0
```

Listing 5: Smoke test with automatic Slurm allocation.

Replace `mypartition` with a valid Slurm partition name on your system. On success, output files appear with names such as `RO_Global.out`, `RO_GONP2.out`, and `slurm-*.out`.

Validate consistency with:

```
bash ../Exec/CheckRun.sh test 0 1 4 3 100
```

The script prints **SUCCESS** or **REPEATING** when runs are consistent; **FAILURE** indicates a configuration or build problem. Adjust `total_stages` and `total_groups` to match your configuration (see Section 4).

4 Running Proteo

Proteo is launched through shell scripts in `Exec/`. All scripts accept **JSON** configuration files (extension `.json`). SAM selects the reader based on file extension.

4.1 Single execution with automatic allocation

`singleRun.sh` submits a Slurm job, computes the required node count from the JSON file, and runs the emulation.

```
bash singleRun.sh config.json partition [outIndex] [repetitions]
                    [valgrind/extrac] [timeLimitSec] [outputDir]

bash singleRun.sh Codes/test.json mypartition 0 1 0
```

Listing 6: `singleRun.sh` usage.

`config.json` Mandatory. Path to the JSON configuration.

`partition` Mandatory. Slurm partition name.

`outIndex` Optional (default 0). Prefix `R{index}` for output files.

`repetitions` Optional (default 1). Number of runs with the same configuration.

`valgrind/extrac` Optional (default 0). 0 = none, 1 = Valgrind, 2 = Extrac tracing.

`timeLimitSec` Optional (default 0). Maximum time in **seconds** for a **single** execution. The script converts this to a Slurm wall-time limit as $(\text{timeLimitSec} \times \text{repetitions})/60 + 1$ minutes. 0 leaves the limit unset.

`outputDir` Optional. Move `R{index}.*` files to this directory after completion.

4.2 Execution with singleRunCostum.sh

`singleRunCostum.sh` does **not** allocate resources itself. It launches Proteo with `mpirun` using the hosts available in the current environment. Parameters match `singleRun.sh` except there is no partition argument. Typical deployment modes:

4.2.1 Local or personal computer

On a single machine (no cluster), invoke the script directly after compiling. Without Slurm, the launcher falls back to the local hostname:

```
bash Exec/singleRunCostum.sh Codes/test.json 0 1 0
bash Exec/singleRunCostum.sh my_config.json 5 10 0 $HOME/results
```

Listing 7: Local run without Slurm.

This is the simplest way to try Proteo on a laptop or workstation.

4.2.2 Cluster with Slurm

Obtain an allocation first (`sbatch` or `srun`), then run the script **inside** that allocation so `SLURM_JOB_NODELIST` is set:

```
# Batch job that wraps the script
sbatch -N 2 -p mypartition --wrap \
    "bash Exec/singleRunCostum.sh Codes/test.json 0 1 0"

# Or interactively after srun / salloc:
bash Exec/singleRunCostum.sh Codes/test.json 0 1 0
```

Listing 8: `singleRunCostum.sh` inside a Slurm allocation.

4.2.3 Multiple machines without an RMS

When several hosts are available but no resource manager is used, MPI must be able to reach every machine (shared filesystem, SSH keys or equivalent). Either:

- Export a comma-separated host list that the launcher can pass to `mpirun -hosts` (the script reads `SLURM_JOB_NODELIST` when set; without Slurm you can still export a host list in that variable), then call `singleRunCostum.sh`; or
- Source `Codes/build/config.txt` and call `mpirun` yourself with an explicit host list.

```
# Option A: host list via environment, then the wrapper script
export SLURM_JOB_NODELIST=host1,host2,host3
bash Exec/singleRunCostum.sh Codes/test.json 0 1 0
```

```
# Option B: direct mpirun after sourcing paths
source Codes/build/config.txt
numP=$(bash Exec/BashScripts/getNumPNeeded.sh Codes/test.json 0)
mpirun -hosts host1,host2,host3 -np $numP $PROTEO_BIN Codes/test.json 0
```

Listing 9: Multi-machine launch without Slurm.

4.3 Combinatorial JSON expansion

To run many concrete configurations from one **complex** JSON file (colon-separated variants), expand first:

```
python3 Exec/PythonCodes/read_multiple_json.py \
  GenConfig/examples/complex.example.json my_run_
```

Listing 10: Expand complex JSON into concrete files.

This writes `my_run.0.json`, `my_run.1.json`, ... into `Desglosed-<date>/` under the current working directory. Each file is sanitized and validated. Then run any expanded file with `singleRun.sh` or `singleRunCostum.sh`.

See Section 6 for variant syntax and GenConfig multi-mode editing.

4.4 Error checking

`CheckRun.sh` verifies that output files for a batch of runs are complete and consistent; it can relaunch failed repetitions.

```
bash CheckRun.sh commonName maxIndex repetitions totalStages totalGroups
  maxIterTime

bash CheckRun.sh test 0 1 4 3 100
```

Listing 11: CheckRun.sh usage.

commonName Prefix shared by output files (e.g. `test` for `R0_*` when index is 0).

maxIndex Highest run index included in the check.

repetitions Repetitions per index.

totalStages Stage count (must be consistent across checked runs).

totalGroups Process groups per run (= `Total_Resizes` + 1).

maxIterTime Maximum valid iteration time (seconds); longer iterations trigger a re-run.

4.5 Intermediate output files

Each run produces:

- `RX_Global.out`: global resize and timing summary.
- `RX_GYNPZ.out`: per-group stage and iteration data.

where X is the run index, Y the group id (0-based), and Z the process count in that group.

Global file contains configuration echo, T_{spawn} , T_{spawn_real} , T_{SR} , T_{AR} , and T_{total} .

Group files contain configuration echo, stage parameters, group resize settings, per-iteration times T_{iter} and T_{stages} , and $Async_iters$ (iterations completed while an asynchronous resize was in progress).

Slurm stdout/stderr is stored separately (e.g. `slurm-12345.out`).

Example for run index 5 with resize from 2 to 10 processes: `R5_Global.out`, `R5_GONP2.out`, `R5_G1NP10.out`.

5 JSON configuration reference

Proteo configuration files are JSON objects with three top-level keys: **general**, **phases**, and **groups**.

```
{
  "general": { ... },
  "phases": [
    {
      "Total_Iters": 1,
      "Total_Stages": 2,
      "stages": [ { ... }, { ... } ]
    }
  ],
  "groups": [
    { "Iters": 10, "Procs": 4, ... }
  ]
}
```

Listing 12: Top-level schema.

Consistency rules:

- `len(phases)` must equal `general.Total_Phases`.
- `len(groups)` must equal `general.Total_Resizes + 1`.
- For each phase, `len(stages)` must equal `Total_Stages` in that phase.

5.1 General parameters

Field	Type	Description
Total_Resizes	integer	Number of resize operations during the run.
Total_Phases	integer	Number of entries in phases .
SDR	number	Count of data elements redistributed synchronously on each resize.
ADR	number	Count of data elements redistributed asynchronously. In combinatorial expansion, values 0–100 may be treated as a percentage of SDR .
Datasize	integer	Size in bytes of each element. Effective sync volume = $\text{SDR} \times \text{Datasize}$; async volume = $\text{ADR} \times \text{Datasize}$.
Rigid	0 or 1	1: insert MPI_Barrier between iterations for precise timing; 0: no barriers.
Capture_Method	0, 1, 2	Aggregate stage times in results: 0=max, 1=mean, 2=median.

Table 2: general object fields.

5.2 Phase and stage parameters

Each element of **phases** describes one emulated application phase.

Field	Scope	Description
Total_Iters	phase	Iterations to execute this phase within the current process group.
Total_Stages	phase	Number of stages in stages .
Stage_Type	stage	Kind of stage (Table 4).
Stage_Time	stage	Target stage duration in seconds (float).
Stage_Bytes	stage	Bytes communicated or transferred (comm/I/O).
Granularity	stage	Problem size / operation count driver (see rules below).
Stage_Time_Capped	stage opt.	0 = complete by operation count; 1 = time cap.
Stage_Identifier	stage opt.	Links IPoint (3) and Wait (4) stages.
Stage_Involved_Procs	stage opt.	I/O only: participating processes (0 = all).

Table 3: Phase and stage fields.

5.2.1 Stage types

Value	Name	Category
0	Monte Carlo (π)	Compute
1	Matrix-vector	Compute
2	Point-to-point	Communication
3	IPoint-to-point (async P2P)	Communication
4	MPI-Wait	Communication
5	Bcast	Communication
6	Allgather	Communication
7	Reduce	Communication
8	Allreduce	Communication
9	I/O write	I/O
10	I/O read	I/O

Table 4: `Stage_Type` values (matches SAM `compute.methods` enum).

5.2.2 Stage validation rules

GenConfig, the expansion pipeline, and SAM emulation expect configurations that satisfy these rules. Using a configuration that violates them is **undefined behaviour** at runtime.

- **Compute (0–1):** `Stage_Time` > 0 and `Granularity` ≥ 1 required. `Stage_Bytes`, `Stage_Identifier`, and `Stage_Involved_Procs` are **unused**.
- **Communication (2–8):** `Stage_Bytes` > 0 **or** `Stage_Time` > 0 required. `Stage_Involved_Procs` is unused.
- **IPoint (3):** `Stage_Identifier` > 0 required. Each IPoint must have at least one Wait stage (type 4) with the same identifier later in the phase.
- **Wait (4):** `Stage_Identifier` > 0 required. A **single** Wait stage may complete **multiple** IPoint stages that share the same identifier (all outstanding requests with that id are waited on together). Duplicate Wait stages with the same identifier are invalid.
- **I/O (9–10):** `Stage_Bytes` > 0 **or** `Stage_Time` > 0; `Stage_Involved_Procs` required (≥ 0).
- **Granularity on non-compute stages:** required (≥ 1) when bytes are unset, or when both bytes and `Stage_Time_Capped=1` are set; optional when bytes are set without time cap. Time-capped stages also require `Stage_Time` > 0.
- If both bytes and time are set on a comm stage, bytes take precedence (warning only in GenConfig).

5.3 Group parameters

Each entry in `groups` configures one process group. Group 0 starts the job; each subsequent group begins after a resize.

Field	Type	Description
<code>Iters</code>	integer > 0	Iterations for this group before next resize (non-final groups; see termination below).
<code>Procs</code>	integer > 0	MPI process count in this group.
<code>FactorS</code>	float ≥ 0	Scalability factor for compute stages (Section 2.7).
<code>Dist</code>	string	"compact" or "spread" physical mapping.
<code>Spawn_Method</code>	0 or 1	0=Baseline, 1=Merge.
<code>Spawn_Strategy</code>	int[]	Spawn strategy indices (Table 5.3.2).
<code>Redistribution_Method</code>	0-3	Data redistribution method (Table 5.3.3).
<code>Redistribution_Strategy</code>	int[]	Exactly one strategy index (empty $\rightarrow$ clear).

Table 5: `groups[]` fields.

Spawn and redistribution settings in group 0 are ignored at job start (processes are created by `mpirun`); they apply from the first resize onward.

5.3.1 Group termination semantics

- **Non-final groups** (group index < `Total_Resizes`): execution stops when `groups[i].Iters` iterations have been executed across the phase loop, which may occur **mid-phase** (before all phases finish).
- **Final group**: `Iters` does **not** truncate the phase list. SAM runs through **all configured phases** until the **last phase completes**. The emulation ends only when that final phase finishes: not merely when a group counter would otherwise trigger a resize.

5.3.2 Spawn strategies

Index	Name
0	Clear (empty / default)
1	Pthread (async spawn thread)
2	Single (rank 0 spawns)
3	Intercomm
4	Multiple
5	Parallel

Table 6: Spawn strategy indices.

5.3.3 Redistribution methods and strategies

Method	Name
0	Baseline / All-to-all (sources)
1	Point to point
2	RMA Lock
3	RMA LockAll

Table 7: `Redistribution.Method` values.

Index	Name
0	Clear
1	Pthread (async)
2	Wait sources
3	Wait targets

Table 8: Redistribution strategy indices.

5.3.4 Strategy combination rules

When $ADR > 0$ (asynchronous redistribution is active):

- **Spawn:** strategies **2 (Single)** or **4 (Multiple)** cannot appear together with **5 (Parallel)** in the same `Spawn.Strategy` array.
- **Redistribution:** methods **2 (RMA Lock)** or **3 (RMA LockAll)** require strategy **1 (Pthread)** or **3 (Wait targets)**.

When $ADR = 0$, only synchronous redistribution runs; the redistribution-strategy constraints above do not apply (spawn strategy choices still affect how targets are created).

5.3.5 Other group validation rules

- `Iters` and `Procs` must be positive integers.
- `FactorS` must be a float ≥ 0 .
- `Redistribution.Strategy` must contain exactly one value (or `[0]` for clear).
- Combinatorial expansion skips combinations where consecutive groups share the same `Procs`.
- When `Procs` uses parallel variants in a group, `FactorS` must provide the same number of parallel values (not an independent Cartesian axis).

5.4 Full example

Listing 13 shows a minimal resize from 2 to 10 processes with three stages per iteration. **SDR** counts elements; with **Datasize=1** the synchronous payload is 10^9 bytes.

5.5 Modeling communication and computation

Communication stages can specify **Stage_Bytes** directly (preferred) or derive volume from **Stage_Time** and network characteristics computed at startup.

Computation stages derive the repeat count **operations** from **Stage_Time**, **Granularity**, **FactorS**, and the measured T_{op} on the current system. These derived values are recomputed when **FactorS** or **Procs** changes after a resize.

6 GenConfig

GenConfig is a lightweight web application in **GenConfig/** for creating, validating, and downloading Proteo JSON configuration files. It implements the same schema and validation rules documented in Section 5.

6.1 Install and launch

```
cd GenConfig
pip install -r requirements.txt
python3 app.py
```

Listing 14: Start GenConfig.

Open <http://127.0.0.1:5000> in a browser. Optional environment variables: **CONFIG_GEN_HOST**, **CONFIG_GEN_PORT**.

6.2 Editor workflow

1. On start, the editor loads a **minimal blank** configuration (one phase, one stage, one group).
2. **New**: in Single mode loads a template from **Codes/test.json** when available; in Multi mode loads a blank complex scaffold.
3. Edit **General**, **Phases** (collapsible, with nested stages), and **Groups** sections.
4. Set **Config file name** for downloads (default **config.json** or **complex.json** in Multi mode).
5. **Validate**: runs structural, stage, and group checks; applies field cleanup (*sanitization*).

```

{
  "general": {
    "Total_Resizes": 1,
    "Total_Phases": 1,
    "SDR": 1000000000,
    "ADR": 0,
    "Datasize": 1,
    "Rigid": 0,
    "Capture_Method": 0
  },
  "phases": [
    {
      "Total_Iters": 1,
      "Total_Stages": 3,
      "stages": [
        {
          "Stage_Type": 0,
          "Stage_Bytes": 0,
          "Stage_Time": 0.16,
          "Granularity": 100000
        },
        {
          "Stage_Type": 8,
          "Stage_Bytes": 8,
          "Stage_Time": 0
        },
        {
          "Stage_Type": 6,
          "Stage_Bytes": 0,
          "Stage_Time": 0.025
        }
      ]
    }
  ],
  "groups": [
    {
      "Iters": 1,
      "Procs": 2,
      "FactorS": 0.5,
      "Dist": "spread",
      "Spawn_Method": 0,
      "Spawn_Strategy": [0],
      "Redistribution_Method": 0,
      "Redistribution_Strategy": [0]
    },
    {
      "Iters": 10,
      "Procs": 10,
      "FactorS": 0.1,
      "Dist": "spread",
      "Spawn_Method": 0,
      "Spawn_Strategy": [0],
      "Redistribution_Method": 0, 21
      "Redistribution_Strategy": [0]
    }
  ]
}

```

Listing 13: Minimal resize example (2 to 10 processes).

Proteo Config-Gen

Build JSON configuration files for SAM / MaM (Phases_Feature schema)

Single
Multi
New
Open file...

Config file name:
complex.json

Validate
Download JSON

▼ General

Total_Resizes

Number of resizes during execution. Groups count = this + 1.

1

Total_Phases

Number of phases (must match phases array length).

2

SDR

Total bytes redistributed synchronously on each resize. Separate alternatives for combinatorial expansion (e.g. 10:20).

1000:3000

ADR

Total bytes redistributed asynchronously on each resize. Separate alternatives for combinatorial expansion (e.g. 10:20).

0:100

Datasize

Size of each redistributed data element (bytes). Separate alternatives for combinatorial expansion (e.g. 10:20).

4

Rigid

If 1, MPI_Barrier is used between iterations for precise time recording; if 0, no barriers between iterations. Separate alternatives for combinatorial expansion (e.g. 10:20).

0

Capture_Method

How stage times are captured in results (0=max, 1=mean, 2=median). Separate alternatives for combinatorial expansion (e.g. 10:20).

0

▼ Phases (2)

Add phase

► Phase 0 — 1 iters, 3 stages

► Phase 1 — 1000 iters, 5 stages

► Groups (2)

Output matches read.json.c. Legacy flat INI [stageN] layouts are not supported here. Group cross-field rules (e.g. Merge spawn strategy) will be added in a future version. Validate/Download apply stage semantic rules and field cleanup. Multi mode uses X:Y colon syntax for combinatorial variants; expand with

Figure 4: GenConfig editor (Single mode).

22

6. **Download JSON**: saves the current configuration locally. Download is never blocked; validation messages are advisory.
7. **Open file...**: load JSON from disk; auto-detects Multi mode when variant strings contain `:`.

Counts stay synchronized automatically:

- `Total_Phases` = number of phase cards.
- `Total_Resizes` = number of groups $- 1$.
- Each phase's `Total_Stages` = number of stages in that phase.

6.3 Single vs Multi mode

Use the toolbar toggle **Single** — **Multi**:

Single mode Edit one concrete configuration. Numeric fields, select boxes for enums, and strategy arrays. Output is a ready-to-run JSON file for `singleRun.sh`.

Multi mode Edit a **complex** configuration with combinatorial variants. Scalar fields accept strings such as `"10:20"`. Strategy fields accept `"0,1:0"` syntax. Validate shows how many valid expanded outputs would be produced. Download saves the complex file for CLI expansion.

6.4 Combinatorial variant syntax

Delimiter	Meaning	Example
<code>:</code>	Cartesian alternative	e.g. <code>"500000:1000000"</code> yields two SDR element counts
<code>,</code>	Values kept together	e.g. <code>"0,1:0"</code> yields <code>[0,1]</code> or <code>[0]</code>

Table 9: Multi-mode variant syntax (INI-compatible convention).

Rules:

- Structural counts (`Total_Phases`, `Total_Resizes`, `Total_Stages`) cannot use variants.
- ADR values 0–100 with variant SDR are percentages (same as legacy INI `correct_adr`).
- Parallel `Procs` / `FactorS` pairs must have matching variant counts.
- Expansion skips combinations with duplicate consecutive `Procs` across groups.

Example complex file: `GenConfig/examples/complex.example.json`.

6.5 Expanding complex JSON

From the repository root:

```
python3 Exec/PythonCodes/read_multiple_json.py \  
  GenConfig/examples/complex.example.json my_run_
```

Listing 15: CLI expansion.

Writes `Desglosed-<date>/my_run_0.json`, `my_run_1.json`, ... Each output passes structural and stage semantic validation.

Then execute any concrete file inside an existing allocation:

```
bash Exec/singleRunCostum.sh Desglosed-2026-07-13/my_run_0.json 0 1 0
```

6.6 Strategy legend in the UI

GenConfig shows spawn and redistribution strategy names (indices **1** and above; **0** means clear and is omitted from the legend):

Spawn: 1=Pthread, 2=Single, 3=Intercomm, 4=Multiple, 5=Parallel.

Redistribution: 1=Pthread, 2=Wait sources, 3=Wait targets.

Leave a strategy field empty for the default clear value [0]. Strategy combination constraints apply when `ADR > 0` (Section 5.3.4).

6.7 Related files

File	Role
<code>app.py</code>	Flask server and API endpoints
<code>schema.py</code>	Field metadata, enums, defaults
<code>validator.py</code>	Structural validation
<code>stage_rules.py</code>	Stage sanitization and semantics
<code>restrictions.py</code>	Group cross-field rules
<code>expand_multi.py</code>	Variant parsing and Cartesian expansion
<code>static/app.js</code>	Browser UI logic
<code>examples/complex.example.json</code>	Sample multi-config

Table 10: GenConfig source files.

See `GenConfig/README.md` for API details (`/api/validate`, `/api/multi/preview`, etc.).

7 Examples

This section walks through typical Proteo workflows with JSON configurations.

7.1 Minimal single run

Use the bundled test configuration `Codes/test.json` (three phases, two resizes, multiple stage types including I/O):

```
cd Results
bash ../Exec/singleRun.sh ../Codes/test.json mypartition 0 1 0
bash ../Exec/CheckRun.sh test 0 1 4 3 100
```

Listing 16: Run test.json with Slurm auto-allocation.

Expected outputs (index 0): `R0_Global.out`, `R0_GONP2.out`, `R0_G1NP4.out`, `R0_G2NP6.out`, plus `slurm-*.out`.

7.2 Custom allocation

Obtain a Slurm allocation first, then run inside it:

```
sbatch -N 2 -p mypartition --wrap \
    "bash Exec/singleRunCostum.sh Codes/test.json 0 5 0"
```

Listing 17: Run inside an sbatch allocation.

This runs five repetitions with output prefix `R0`. Use a different `outIndex` (2nd argument) to distinguish batches: `R5_*`, `R6_*`, etc.

7.3 Creating a config with GenConfig

1. Start GenConfig (`python3 app.py` in `GenConfig/`).
2. Stay in **Single** mode. Click **New** to load the `test.json` template.
3. Adjust **Procs** and **Iters** in the groups section.
4. Click **Validate**, then **Download JSON** as `my_experiment.json`.
5. Inside an allocation: `bash Exec/singleRunCostum.sh my_experiment.json 0 1 0`.

7.4 Simple resize example

Listing 13 (Section 5) defines one phase with Monte Carlo + Allreduce + Allgather, resizing from 2 to 10 processes after one source-group iteration. Save it as `simple_resize.json` and run:

```
bash Exec/singleRunCostum.sh simple_resize.json 0 1 0
```

7.5 Multi-phase application

Phase 0 of Codes/test.json mixes I/O read, Bcast, and compute:

```
{
  "Total_Iters": 1,
  "Total_Stages": 3,
  "stages": [
    {
      "Stage_Type": 10,
      "Stage_Bytes": 0,
      "Stage_Involved_Procs": 2,
      "Stage_Time": 4,
      "Granularity": 1000
    },
    { "Stage_Type": 5, "Stage_Bytes": 1000, "Stage_Time": 0 },
    {
      "Stage_Type": 0,
      "Stage_Bytes": 0,
      "Stage_Time": 1,
      "Granularity": 10000
    }
  ]
}
```

Listing 18: Phase 0 of Codes/test.json.

Phase 1 runs 22 iterations with compute + two Allreduce stages + Allgather; phase 2 performs a single large I/O write. This illustrates how phases chain different stage sequences within the same process group.

7.6 Combinatorial study

Variant fields in GenConfig/examples/complex.example.json:

```
"SDR": "500000:1000000",
"ADR": "0:50",
"Procs": "2:4",
"FactorS": "1:1"
```

Listing 19: Variant fields in complex.example.json.

Workflow:

1. Edit variants in GenConfig Multi mode (or edit the file directly).
2. Validate to see the predicted number of valid combinations.
3. Download as complex.json.
4. Expand: `python3 Exec/PythonCodes/read_multiple_json.py complex.json sweep_.`

5. Run each `Desglosed-*/sweep.N.json` with `singleRun.sh`, varying partition and output index as needed.

7.7 Conjugate-gradient-like profile

A CG-like iteration can be approximated with one compute stage plus several communication stages (see Listing 13 for structure). Use matrix-vector compute (`Stage_Type`: 1), small Allreduce messages (`Stage_Bytes`: 8), and a large Allgather (`Stage_Bytes` set explicitly). Tune `Granularity` and `Stage_Time` to match target iteration time on a reference process count, then set `FactorS` per group for scaling behaviour.

7.8 Error checking after a batch

After multiple runs sharing prefix `exp`:

```
bash Exec/CheckRun.sh exp 9 5 4 3 120
```

Checks indices 0 through 9, five repetitions each, expecting four stages and three groups, rejecting iterations longer than 120 seconds.

8 Output analysis

Simulation runs produce intermediate text files (Section 4.5). This section describes the Python tooling available today for turning those files into structured data.

8.1 Analysis scripts

Post-processing is handled by scripts in `Analysis/`. The recommended pipeline is:

1. `MallTimes.py`: primary entry point. Reads raw Proteo `.out` files and produces the base dataframe(s) / CSV used by downstream steps.
2. `joinDf.py`: when experiments were split into multiple dataframe files, merges them into one (`python3 joinDf.py df1.pkl df2.pkl outputName`).
3. `CreateIterDataframe.py`, `CreateStageDataframe.py`, `CreateResizeDataframe.py`: derive specialised views (per-iteration, per-stage, per-resize) from the `MallTimes` output.

Example:

```
python3 Analysis/MallTimes.py R /path/to/results/ data_global.csv
```

Listing 20: Build base dataframe from result files.

R Common prefix of Proteo output files (e.g. R matches `R0_Global.out`, `R1_Global.out`, ...).

`/path/to/results/` Directory containing result subdirectories or flat output files.

`data_global.csv` Output path for the aggregated data.

Requirements: Python 3 with NumPy and Pandas.

8.2 Notebook analyser (WIP)

`Analysis/new_analyser.ipynb` is a work-in-progress Jupyter notebook for statistical summaries and plots. It is **not yet suitable for general use** by other users (hard-coded paths, evolving API). Future manual revisions will document it once stabilised.

8.3 Recommended practice

- Run `CheckRun.sh` before analysis to filter or relaunch inconsistent repetitions.
- Use at least five repetitions per configuration when comparing strategies statistically.
- Keep `Rigid=1` when precise iteration timing matters; use `Rigid=0` for performance-oriented studies where barrier overhead should be avoided.

For column-level CSV documentation, see Appendix A.

A Output CSV files format

In this appendix is described the format of each final output file, in order to allow users to analyze by themselves the obtained results.

Each of the rows of file `data_global.csv` represents a complete execution of a malleable application. Columns can be grouped according to their meaning. On the one hand, there are those showing information on the resizing, on the computation in each iteration and the results obtained.

- Columns related to resize configuration:
 - `Total_Resizes`: It is an integer that indicates number of performed resizes (`Total_Resizes` in configuration file).
 - `Total_Groups`: It is an integer indicating the number of processes groups for the malleable execution. It is equal to `Total_Resizes+1`. Each of the following six columns is a list of data showing the concrete information for each group in a resize. They are a ordered array of integers with size `Total_Groups`.

- Groups: Number of processes in a group.
 - Dist: Physical distribution of processes in a group. Its possible values are Compact(0) and Spread(1).
 - Spawn_Method: Spawn method for a resize. Its possible values are Baseline(0) and Merge(1).
 - Spawn_Strategy: Spawn strategy for a resize. Its possible values are None(1), Single(3), Asynchronous(2) and All(6).
 - Redistribution_Method: Redistribution method for each resize. Array of integers with size *Total_Groups*. Its possible values are Alltoall(0), Point-to-point(1), RMA-Lock(2) or RMA-LockAll(3).
 - Redistribution_Strategy: Redistribution strategy for each resize. Array of integers with size *Total_Groups*. Its possible values are None(1), Threads(2).
 - SDR: Number of data elements redistributed synchronously in each redistribution stage. Integer. Multiply by **Datasize** for bytes moved.
 - ADR: Number of data elements redistributed asynchronously in each redistribution stage. Integer. Multiply by **Datasize** for bytes moved.
 - DR: Total elements redistributed in each redistribution stage. Is equal to $SDR + ADR$. Integer.
- Columns related to application type:
 - Granularity: It is an integer that indicates problem size for computation stages.
 - FactorS: Speedup factor to emulate how the performance of computation stages are affected by the number of processes. Ordered array of Integers of size *Total_Groups*.
 - Total_Stages: It is an integer that represents number of stages in an iteration. The three following columns contain concrete information about each of stages. They will be an ordered array of size *Total_Stages*.
 - Stage_Types: Each element indicates stage type (computation/communication). Its possible values are PI_Computation(0), Matrix_Computation(1), Point_communication(2), Bcast_communication(3), Allgatherv_communication(4), Reduce_communication(5) and Allreduce_communication(6).
 - Stage_Times: Each element represents the stage time.
 - Stage_Bytes: Each element indicates bytes communicated in this stage. For computation stages is always 0.
 - Results columns:
 - Iters: It is an ordered array of integers with size *Total_Groups* that contains executed iterations by each group.

- *Asynch_Iters*: It is an ordered array of integers with size *Total_Groups* that represents the total iterations executed by the parents while an asynchronous resize was taking place.
- *T_iter*: Array of arrays (2 levels) which contains the times of each iteration. The upper array refers to the iterations of each group and has a size of *Total_Groups*. The inner array contains floats, and the size of the array with index *i* is *Iters[i]*, where *i* refers to a group of processes. Therefore, to check the iteration time of the last iteration in the first group of processes it will be accessed as $T_{iter}[0][Iters[0] - 1]$.
- *T_stages*: Array of 3 levels which contains the times of each performed stage in the execution. The first level contains arrays of groups and is of size *Total_Groups*. The second level contains an array of iterations, and the size of the array with index *i* is *Iters[i]*, where *i* refers to a group of processes. Lastly, the third level stores for a specific iteration, an array of floats values where each one represents the time of a stage in the iteration. Has a size of *Total_Stages*. Therefore, to check the stage time of the last stage in the last iteration of the first group of processes it will be accessed as $T_{iter}[0][Iters[0] - 1][Total_Stages - 1]$.
- *T_spawn*: Ordered array of spawn times for each resize. Refers to the time T_{spawn} in the monitoring module (Subsection 4). Array of floats with size *Total_Resizes*.
- *T_spawn_real*: Ordered array of real spawn times for each resize. Refers to the time T_{spawn_real} in the monitoring module (Subsection 4). Array of floats with size *Total_Resizes*.
- *T_SR*: Ordered array of synchronous data redistribution times for each resize. Refers to the time T_{SR} in the monitoring module (Subsection 4). Array of floats with size *Total_Resizes*.
- *T_AR*: Ordered array of asynchronous data redistributions times for each resize. Refers to the time T_{AR} in the monitoring module (Subsection 4). Array of floats with size *Total_Resizes*.
- *T_total*: Total execution time of the application. Float value.

The other two files generated by *analyser.sh* script, *data_malleability.csv* and *data_iterations.csv*, are focused on helping the analysis of behaviour of malleable application while resizing is being performed above when it is carried out asynchronously. In both files, each row represents a different malleable running and its columns are a subset of those described in the previous file.

The second data set *data_malleability.csv*, is intended to facilitate the analysis of different configurations in malleability only, where all the execution data is not needed. Each row represents a single resize. This data set has the next columns:

- Resize configuration columns:

- NP: Number of parent processes before the resize. Integer.
- NC: Number of children processes after the resize. Integer.
- Dist: Physical distribution of processes for the parents and children. Array of two integers. Its possible values are Compact(0) and Spread(1).
- Spawn_Method: Spawn method used for the resize. Integer. Its possible values are Baseline(0) and Merge(1).
- Spawn_Strategy: Spawn strategy used for the resize. Integer. Its possible values are None(0), Asynchronous(1), Single(2), All(3).
- Redistribution_Method: Redistribution method for each resize. Integer. Its possible values are Alltoall (0), Point-to-point(1), RMA-Lock(2) or RMA-LockAll(3).
- Redistribution_Strategy: Redistribution strategy for each resize. Integer. Its possible values are None(1), Threads(2).
- SDR: Number of data elements redistributed synchronously in the redistribution stage. Integer. Multiply by **Datasize** for bytes moved.
- ADR: Number of data elements redistributed asynchronously in the redistribution stage. Integer. Multiply by **Datasize** for bytes moved.
- DR: Total elements redistributed in the redistribution stage. Is equal to $SDR + ADR$. Integer.
- Compute configuration columns:
 - Total_Stages: Number of stages. Integer.
 - Granularity: Problem size for computation stages. Integer.
 - FactorS: Speedup factor to emulate how the performance of computation stages are affected by the number of processes at column *NP*. Integer.
 - Stage_Type: Ordered array of stages types for each stage of the iterations. Array of integer with size *Total_Stages*. Its possible values are PI_Computation(0), Matrix_Computation(1), Point_communication(2), Bcast_communication(3), Allgatherv_communication(4), Reduce_communication(5) and Allreduce_communication(6).
 - Stage_Time: Ordered array of stage times for each stage of the iterations. Array of integer with size *Total_Stages*.
 - Stage_Bytes: Ordered array of bytes communicated for each stage of the iterations. Array of integer with size *Total_Stages*.
- Results columns:
 - Iters: Quantity of performed iterations by the parents. Is the sum of *Asynch_Iters* and synchronous iterations. Integer.

- *Asynch_Iters*: Quantity of performed iterations by the parents while an asynchronous resize was taking place. Integer.
- *T_{iter}*: Array of times for each performed iteration in this group. The array contains floats and has a size of *Iters* elements. Therefore, to check the iteration time of the last iteration it will be accessed as *T_{iter}*[*Iters* – 1].
- *T_{stages}*: Array of 2 levels which contains the times of each performed stage by the group. The first level contains an array of iterations, with an array size of *Iters*. On the other hand, the second level stores for a specific iteration, an array of floats values where each one represents the time of a stage in the iteration. Has a size of *Total_Stages*. Therefore, to check the stage time of the last stage in the last iteration it will be accessed as *T_{stages}*[*Iters* – 1][*Total_Stages* – 1].
- *T_{spawn}*: Spawn times for the resize. Refers to the time *T_{spawn}* in the monitoring module (Subsection 4). Float value.
- *T_{spawn_real}*: Real spawn times for the resize. Refers to the time *T_{spawn_real}* in the monitoring module (Subsection 4). Float value.
- *T_{SR}*: Synchronous data redistribution time for the resize. Refers to the time *T_{SR}* in the monitoring module (Subsection 4). Float value.
- *T_{AR}*: Asynchronous data redistribution time for the resize. Refers to the time *T_{AR}* in the monitoring module (Subsection 4). Float value.

The final data set *data_iterations.csv* is a simplification of the previous data sets to focus an analysis on how each iteration has been performed. Each row represents a single iteration. This data set has the next columns:

- NP: Number of processes. Integer.
- Is_Dynamic: Whether the group of processes is the first one (False) or has been spawned during execution (True). Boolean.
- The next values should be ignored if *Asynch_Iters* is False. Resize configuration columns:
 - NC: Number of children processes after the resize. Integer. Default value is *None* if *Asynch_Iters* is False.
 - Dist: Physical distribution of processes for the parents and children. Array of two integers. Its valid values are Compact(0) and Spread(1). Default value for second element is (-1) if *Asynch_Iters* is False.
 - Spawn.Method: Spawn method used for this group and the next one. Array of two integers. Its possible values are Baseline(0) and Merge(1). Default value for first element is (0) if *Is_Dynamic* is False. Default value for second element is (-1) if *Asynch_Iters* is False.

- *Spawn_Strategies*: Spawn strategy used for this group and the next one. Array of two integers. Its possible values are *None*(0), *Asynchronous*(1), *Single*(2), *All*(3). Default value for first element is (1) if *Is_Dynamic* is False. Default value for second element is (-1) if *Asynch_Iters* is False.
- *Redistribution_Method*: Redistribution method used for this group and the next one. Array of two integers. Its possible values are *Alltoall* (0), *Point-to-point*(1), *RMA-Lock*(2) or *RMA-LockAll*(3). Default value for first element is (0) if *Is_Dynamic* is False. Default value for second element is (-1) if *Asynch_Iters* is False.
- *Redistribution_Strategy*: Redistribution strategy used for this group and the next one. Array of two integers. Its possible values are *None*(1), *Threads*(2). Default value for first element is (1) if *Is_Dynamic* is False. Default value for second element is (-1) if *Asynch_Iters* is False.
- *SDR*: Number of data elements redistributed synchronously in the redistribution stage. Integer. Default value is (-1) if *Asynch_Iters* is False.
- *ADR*: Number of data elements redistributed asynchronously in the redistribution stage. Integer. Default value is (-1) if *Asynch_Iters* is False.
- *DR*: Total elements redistributed in the redistribution stage. Integer. Is equal to $SDR + ADR$. Default value is (-1) if *Asynch_Iters* is False.
- Compute configuration columns:
 - *Total_Stages*: Number of stages. Integer.
 - *Granularity*: Problem size for computation stages. Integer.
 - *FactorS*: Speedup factor to emulate how the performance of computation stages are affected by the number of processes at column *NP*. Integer.
 - *Stages_Type*: Ordered array of stage types for each stage of the iterations. Array of integer with size *Total_Stages*. Its possible values are *PI.Computation*(0), *Matrix.Computation*(1), *Point_communication*(2), *Bcast_communication*(3), *Allgatherv_communication*(4), *Reduce_communication*(5) and *Allreduce_communication*(6).
 - *Stage_Time*: Ordered array of stage times for each stage of the iterations. Array of integer with size *Total_Stages*.
 - *Stage_Bytes*: Ordered array of bytes communicated for each stage of the iterations. Array of integer with size *Total_Stages*.
- Results columns:

- *Asynch_Iters*: Indicates whether this iteration has been performed normally (False) or during an asynchronous resize (True). Boolean.
- *T_{iter}*: Iteration time. Float value
- *T_{stage}*: Array of each stage time in the iteration. Array of floats with size *Total_Stages*. Therefore, to check the stage time of the last stage in the iteration it will be accessed as $T_{stages}[Total_Stages - 1]$.